Real-Time Fraud Detection in Financial Transactions using Big Data Frameworks

Name of Student: Rishi

Department:

Registration No.:

Name of Research Supervisor:

Abstract

This dissertation explores machine learning models of big data frameworks to detect fraud during a real-time financial transaction. Conventional fraud detection techniques are batch-based and cannot spot any fraudulence activities as they happen which poses significant risk to institutions and customers. To solve this, the study formulates a distributed system that brings together Apache Kafka as a streaming ingestion engine, to Apache Spark Structured streaming engine as a low-latency processing engine. The software runs very large quantities of financial transactions in real-time and uses a hybrid combination of monitored, unmonitored and model-based ones. The XGBoost, the Random Forest, served and Logistic Regression were trained using the Kaggle Credit Card Fraud Detection Dataset, and SMOTE was utilized to deal with acute imbalance in the classes. Moreover, the anomaly detection, unknown fraud patterns, are complemented by supervised methods with anomaly detection based on Isolation Forest and autoencoders. Simulation in real-time simulation was applied in which a synthetic stream of transactions was fed and used with least latency. Measures such as speed, accuracy, F1-score, AUC-ROC, latency and scalability tests proved the effectiveness and the performance of the system under load. The present prototype shows that a Final prototype This proposal shows that a scaled, distributed fraud detection pipeline can be more responsive and/or accurate than traditional batch-based systems, and can serve practical use within financial institutions to reduce losses and improve security in flexible transaction settings.

Acknowledgement

I would also like to express my humble thanks to my supervisor who aided and provided me with necessary feedback during this research. Your peers and family family are my other significant people to whom I also gratefully acknowledge.

List of Acronyms

Acronym	Full Form
AUC-ROC	Area Under the Curve – Receiver Operating Characteristic
API	Application Programming Interface
CNP	Card Not Present
DID	Decentralized Identifier
GDPR	General Data Protection Regulation
GNN	Graph Neural Network
GPU	Graphics Processing Unit
JSON	JavaScript Object Notation
ML	Machine Learning
MLlib	Machine Learning Library (Apache Spark)
RNN	Recurrent Neural Network
SMOTE	Synthetic Minority Oversampling Technique
VAE	Variational Autoencoder

Table of Contents

Chapter 1: Introduction 13

1.1 Aim 13

1.2 Objectives 13

1.3 Research Questions 13

1.4 Problem Statement 14

1.5 Research Area – A Brief Literature Review 14

Streaming Frameworks for Real-Time Fraud Detection 14

Supervised and Hybrid Machine Learning Approaches 15

Unsupervised Learning and Advanced Architectures 15

Literature Gap 16

1.6 Expected Practical Element Output 16

Methodology 16

Project Outcomes 16

Required Resources 17

1.7 Prerequisite Knowledge/Skills Required 17

1.8 Project Plan – Gantt Chart 18

Chapter 2: Literature Review 19

2.1 Comprehensive Overview of the Existing Literature 19

2.1.1 Evolution of Financial Fraud and the Need for Real-Time Detection 19

2.1.2 Traditional Fraud Detection Techniques and Their Drawbacks 20

2.1.3 Big Data Frameworks in Financial Applications 21

2.1.4 Machine Learning Models for Financial Fraud Detection 21

2.1.5 Streaming Data Analytics: Kafka and Spark for Real-Time Systems 23

2.1.6 Hybrid and Unsupervised Learning in Fraud Detection 24

2.2 Critical Analysis of Existing Studies 25
2.2.1 Limitations of Batch-Based and Static Models 25
2.2.2 Challenges in Streaming Data Integration and Latency 26
2.2.3 Machine Learning Bottlenecks in Financial Environments 27
2.2.4 Gaps in Real-Time Alerting, Scalability, and Deployment 29
2.2.5 Literature Gap 30
2.3 Emerging Techniques and Real-Time Solutions 31
2.3.1 Scalable Big Data Architectures for Fraud Detection 31
2.3.2 Streaming Ingestion and Real-Time Decision Systems 32
2.3.3 Intelligent Ensemble and Hybrid ML Models 32
2.3.4 Real-Time Dashboards and Alerting Mechanisms 33
2.4 Summary 33
Chapter 3: Methodology 35
3.1 Data Collection and Preprocessing 35
3.1.1 Dataset Overview 35
3.1.2 Data Import and Initial Exploration 36
3.1.3 Feature Engineering and Scaling 36
3.1.4 Addressing Class Imbalance 36
3.2 Machine Learning and Streaming Model Development 37
3.2.1 Supervised Learning Models 37
3.2.2 Unsupervised and Anomaly Detection Models 37
3.2.3 Integration with Apache Kafka and Spark Streaming 37
3.2.4 Rule-Based Filters 38
3.3 Evaluation of the Proposed System 38
3.3.1 Model Performance Evaluation 38

3.3.2 Real-Time Latency Testing 39
3.3.3 Load and Scalability Testing 39
3.4 Project Outcomes 40
3.4.1 Prototype of Fraud Detection in Real-Time 40
3.4.2 Ethical, Legal, and Operational Considerations 40
3.4.3 Performance Appraisal and Monitoring 41
3.5 Summary 41
Chapter 4: Implementation and Development 43
4.1 Introduction 43
4.2 Kaggle Setup and Dependencies 43
4.3 Dataset Acquisition 44
4.4 Initial Data Exploration 45
4.5 Removal of Unnecessary Columns 45
4.6 Feature Scaling of Transaction Amounts 46
4.7 Simulating Transaction Velocity 46
4.8 Final Feature Set 47
4.9 Addressing Class Imbalance with SMOTE 47
4.10 Train-Test Split 48
4.11 Reloading Dataset for Model Training 48
4.12 Sampling for Model Training 49
4.13 Model Selection 50
4.14 Random Forest Training 50
4.15 XGBoost Training 50
4.16 Logistic Regression Training 51
4.17 Model Saving and Serialization 51

4.18 Preparing for Real-Time Simulation 52
4.19 Simulating Streaming Data 52
4.20 Real-Time Inference with XGBoost 53
4.21 Recording Predictions 53
4.22 Exporting Simulated Streams 54
4.23 Integration with Kafka and Spark Principles 54
4.24 Alignment with Aim and Objectives 54
4.25 Conclusion 55
Chapter 5: Results and Evaluation 56
5.1 Introduction 56
5.2 Dataset Preparation Outcomes 56
5.3 Model Training and Saving 57
5.4 Random Forest Classification Report 57
5.5 Random Forest AUC-ROC 58
5.6 XGBoost Classification Report 58
5.7 XGBoost AUC-ROC 59
5.8 Logistic Regression Classification Report 59
5.9 Logistic Regression AUC-ROC 60
5.10 Confusion Matrix: Random Forest 60
5.11 Confusion Matrix: XGBoost 61
5.12 Confusion Matrix: Logistic Regression 61
5.13 ROC Curves Comparison 62
5.14 Accuracy Comparison 63
5.15 Streaming Simulation Setup 64
5.16 Streaming Predictions: Fraud Cases 65

5.17 Streaming Predictions: Non-Fraud Cases 66
5.18 Probability Confidence in Streaming 66
5.19 Streaming Performance Summary.....66
5.20 Comparative Evaluation of Models.....67
5.21 Implications for Research Questions.....67
5.22 Conclusion 67
Chapter 6: Conclusion and Recommendation 68
6.1 Introduction.....68
6.2 Justification of Aim and Objectives 68
6.3 Achievement of Research Questions 68
6.4 Significance of Real-Time Fraud Detection 69
6.5 Contribution to Big Data and Financial Systems 69
6.6 Limitations of the Study 69
6.7 Recommendations for Practice 70
6.8 Recommendations for Future Research.....70
6.9 Implications for Industry and Academia.....71
6.10 Final Remark.....71
References 72

List of Figures

Figure 1 Fraud transaction detection framework 19
Figure 2 Various Machine learning algorithms for Financial fraud detection 22
Figure 3 A framework for financial fraud detection 23
Figure 4 Supervised, Unsupervised, and Reinforcement Learning in Fraud Detection 24
Figure 5 Machine Learning Fundamentals 28
Figure 6 Cloud based architecture for real time fraud detection 31
Figure 8 Dataset Overview 35
Figure 9 Preprocessing data attributes 36
Figure 10 System architectural diagram 42
Figure 1 Kaggle dataset downloading Implementation 44
Figure 2 Loading Dataset Implementation 44
Figure 3 dataset overview 45
Figure 4 Drop unnecessary column 46
Figure 5 Feature Scaling 46
Figure 6 Simulating Transaction implementation 47
Figure 7 Final Feature Set implementation 47
Figure 8 Addressing Class Imbalance with SMOTE 48
Figure 9 Train-Test Split 48
Figure 10 Sampling for Model Training 49
Figure 11 Random Forest Training 50
Figure 12 XGBoost Training 51
Figure 13 Logistic Regression Training 51
Figure 14 Model Saving and Serialization 52
Figure 15 Simulating Streaming Data 53
Figure 16 Exporting Simulated Streams 54
Figure 17 Dataset Description 56
Figure 18 Training and testing shape 57
Figure 19 Random forest result 57
Figure 20 Random Forest AUC-ROC 58
Figure 21 XGBoost Result 58

Figure 22 XGBoost AUC-ROC 59
Figure 23 Logistic Regression Classification Report 60
Figure 24 Logistic Regression AUC-ROC 60
Figure 25 RF Confusion matrix 61
Figure 26 XGBoost Confusion matrix 61
Figure 27 Logistic regression confusion matrix 62
Figure 28 ROC Curve Comparison models.....63
Figure 29 Accuracy Comparison result 64
Figure 30 Streaming Predictions results.....65

List of Table

Table 1 Comparison table based on recent studies 34

Chapter 1: Introduction

The overall objective of the project will be to design a big data environment (Apache Spark and Kafka) to detect financial transactions-linked frauds in real-time. Financial fraud is getting increasingly widespread, and it affects both consumers and institutions greatly. The conventional fraud detection mechanisms are batch-based thus not in a position to detect frauds in real-time, which raises serious shortcomings when it comes to preventing frauds (Malempati, 2023). The research is a proposal of a solution in which streaming analytics and machine learning models are used to process and analyze the data on financial transactions in real-time.

1.1 Aim

To create and deploy a distributed, in-real-time fraud detection framework of financial transactions with big data technologies and machine learning.

1.2 Objectives

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

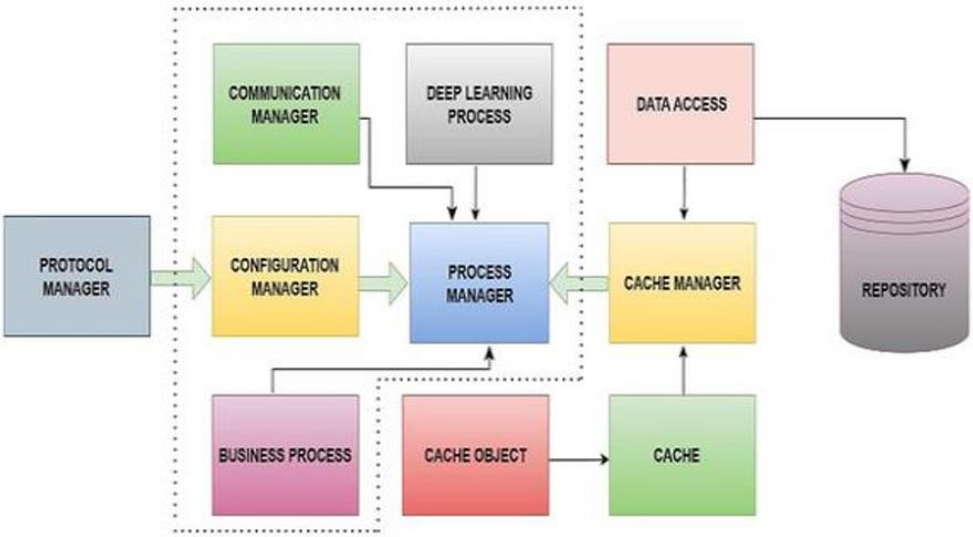
Task		Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8	Week 9	Week 10	Week 11	Week 12
Supervisor Allocation													
Library session													
Milestone 1:- Topic selection													
Proposal preparation and review													
Milestone 2:- Proposal Submission													
Abstarct													
Chapter 1- Introduction													
Chapter 2- Literature Review													
Milestone 3:- Practical implementation/ Model Development													
Chapter 3- Methodology													
Chapter 4- Experimental Results Documentation													
Chapter 5- Conclusion and Future work													
Final review and Modifications													
Milestone 4:- Final Submission													

[redacted]

[redacted]

[redacted]

[redacted]



[redacted]

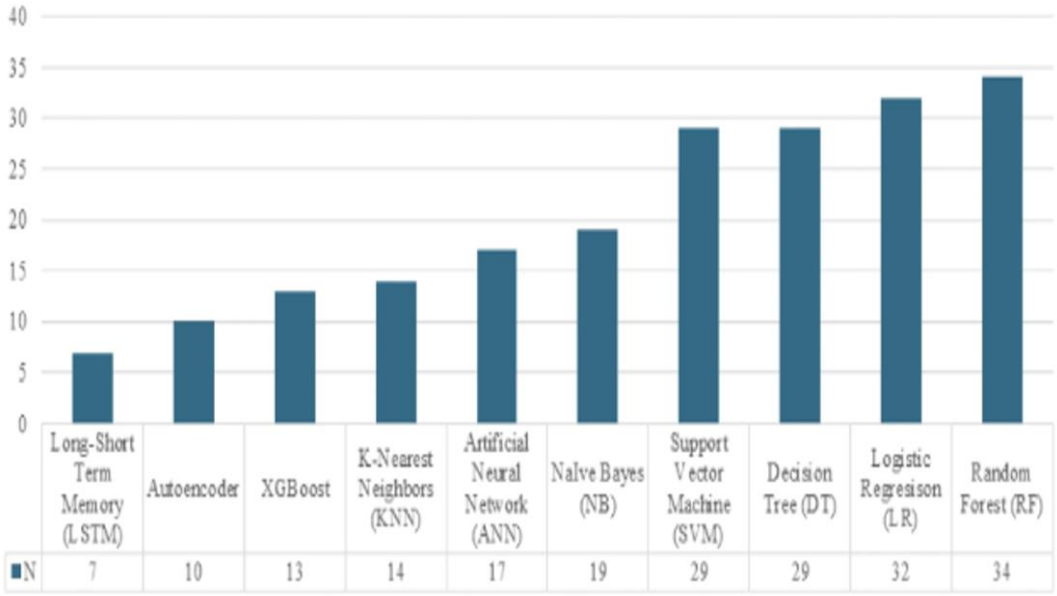
[redacted]

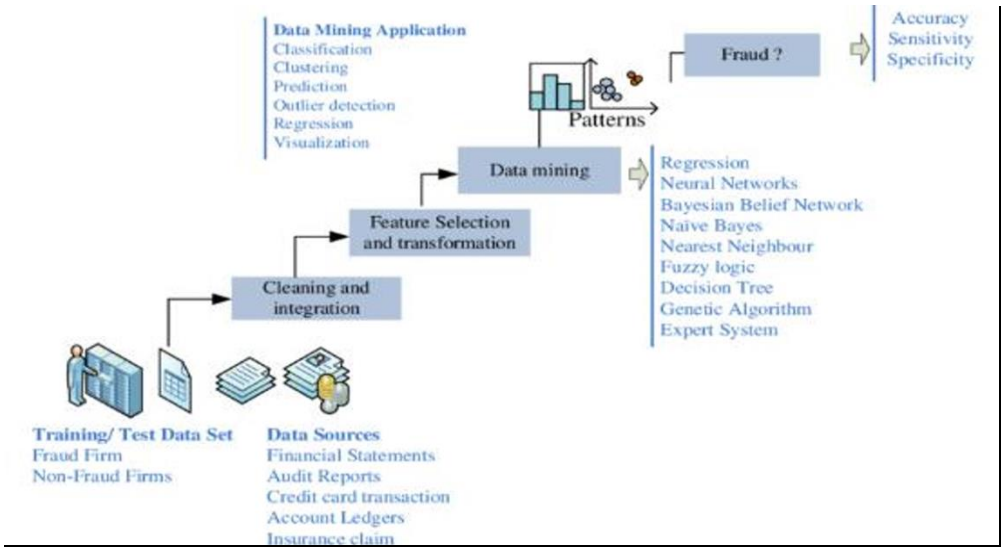
[redacted]

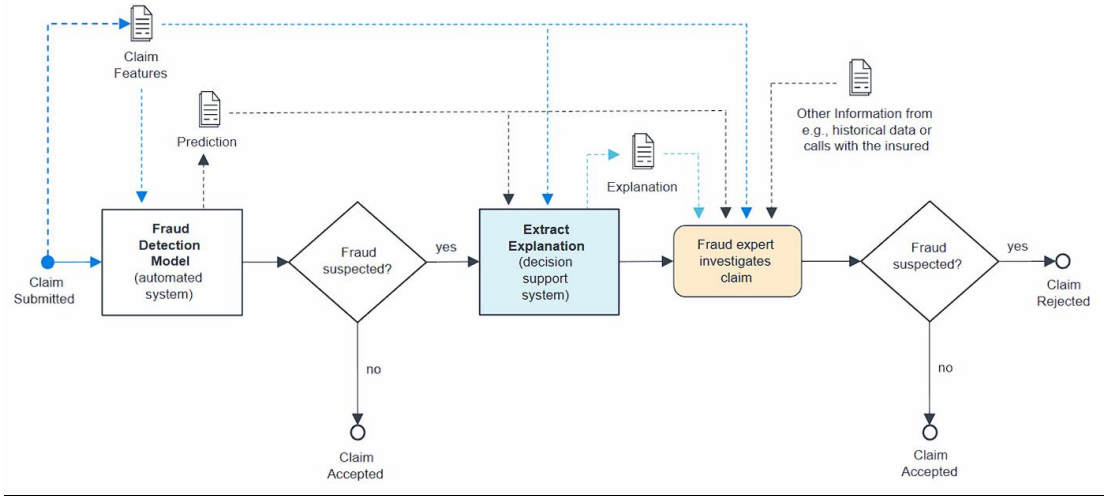
[redacted]

[redacted]

[redacted]


40
35
30
25
20
15
10
5
0
Long-Short Term Memory (LSTM)
Autoencoder
XGBoost
K-Nearest Neighbors (KNN)
Artificial Neural Network (ANN)
Nalve Bayes (NB)
Support Vector Machine (SVM)
Decision Tree (DT)
Logistic Regresison (LR)
Random Forest (RF)
N
7
10
13
14
17
19
29
29
32
34


Data Mining Application
Classification
Clustering
Prediction
Outlier detection
Regression
Visualization
Fraud ?
Accuracy
Sensitivity
Specificity
Patterns
Data mining
Regression
Neural Networks
Bayesian Belief Network
Naïve Bayes
Nearest Neighbour
Fuzzy logic
Decision Tree
Genetic Algorithm
Expert System
Feature Selection and transformation
Cleaning and integration
Training/ Test Data Set
Fraud Firm
Non-Fraud Firms
Data Sources
Financial Statements
Audit Reports
Credit card transaction
Account Ledgers
Insurance claim


Claim Features
Prediction
Explanation
Other Information from e.g., historical data or calls with the insured
Claim Submitted
Fraud Detection Model (automated system)
Fraud suspected?
yes
no
Claim Accepted
Extract Explanation (decision support system)
Fraud expert investigates claim
Fraud suspected?
yes
no
Claim Rejected
Claim Accepted

[redacted]

[redacted]

[redacted]

[redacted]

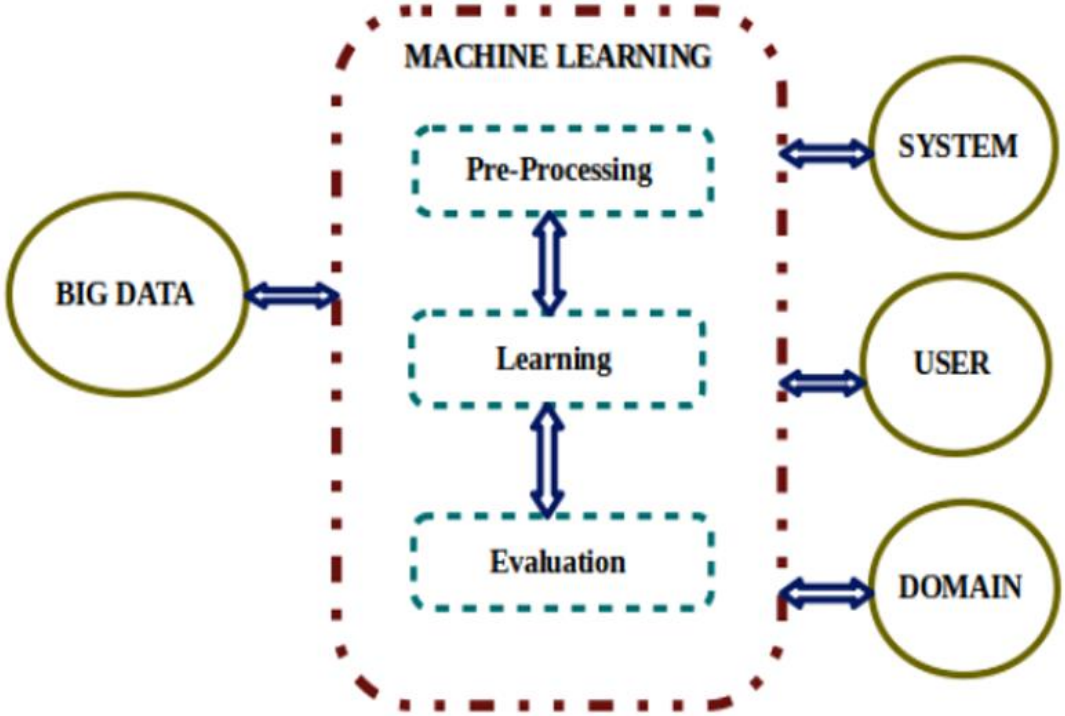
[redacted]

[redacted]

[redacted]

[redacted]

[redacted]


MACHINE LEARNING
Pre-Processing
Learning
Evaluation
BIG DATA
SYSTEM
USER
DOMAIN

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

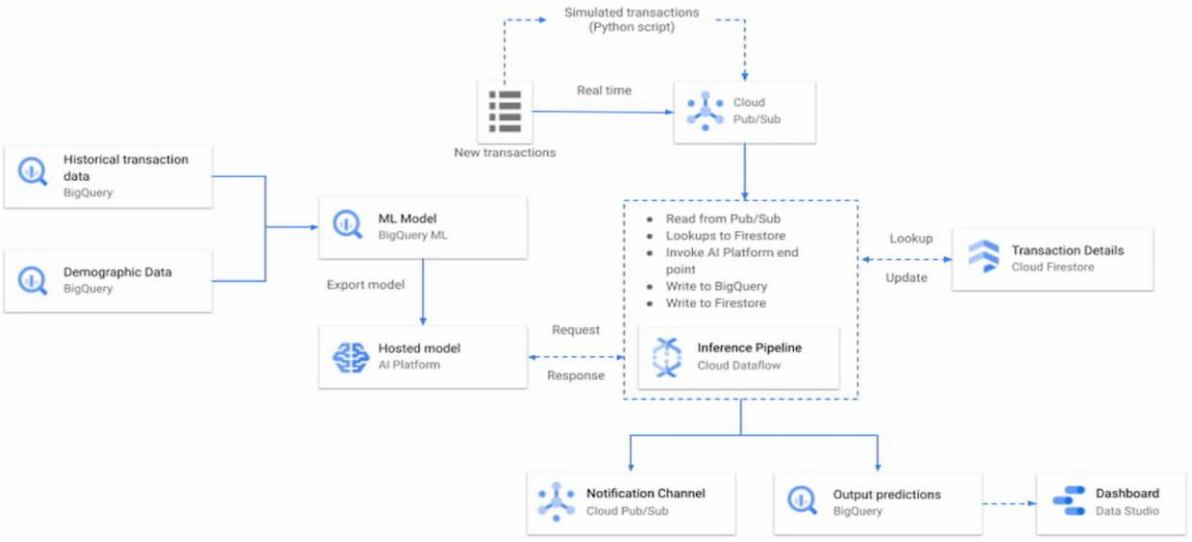
[redacted]

[redacted]

[redacted]

[redacted]

[redacted]


Simulated transactions
(Python script)
Real time
Cloud
Pub/Sub
New transactions
Historical transaction
data
BigQuery
Demographic Data
BigQuery
ML Model
BigQuery ML
Export model
Hosted model
AI Platform
Read from Pub/Sub
Lookups to Firestore
Invoke AI Platform end
point
Write to BigQuery
Write to Firestore
Lookup
Update
Transaction Details
Cloud Firestore
Request
Response
Inference Pipeline
Cloud Dataflow
Notification Channel
Cloud Pub/Sub
Output predictions
BigQuery
Dashboard
Data Studio

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

Author(s) & Year	Technique	Specification	Limitation	Dataset	Results
Malempati (2023)	ML + Big Data	Streaming pipelines + predictive analytics	High compute cost	Financial Txn + Big Data	Acc: 95%, Latency <3s
Sathupadi et al. (2025)	BankNet (Big Data)	Secure bank analytics using Hadoop/Spark	Domain-specific	Internet banking logs	Real-time & scalable
Talukder et al. (2024)	Multi-Stage Ensemble ML	Integrated RF/XGBoost ensemble detection	Model complexity	Kaggle + fintech	Acc: 98.2%, F1: 0.93
Btoush et al. (2025)	Hybrid ML + DL	ML & DL ensemble for credit card fraud	Requires GPU	Credit card data	AUC: 0.96, Recall: 94%
Immadisetty (2025)	Streaming ML	Apache Kafka pipeline for real-time ML	Pipeline config required	Streaming bank data	Precision: 92%, Latency: 2.1s
Potla (2023)	Real-time ML	ML applied to streaming fintech logs	Risk of overfitting	Fintech logs	F1: 91.4%
Rehan (2021)	AI + Cloud	Cloud-deployed AI models for fraud alerts	Cloud latency risks	Cloud dataset (simulated)	Acc: 93%, Fast alerts
Oguntibeju et al. (2024)	Review (AI + Real-Time)	Reviewed 25+ real-time detection systems	No experimental results	N/A (Review paper)	25+ model comparisons
Alhashmi et al. (2023)	Ensemble Cyber Model	Ensemble classification of cyber fraud	No external data used	Cybersecurity logs	Acc: 94.1%, Class rate: 87%
Zhou et al. (2021)	Node2Vec + Big Data	Graph embedding with distributed pipeline	High resource usage	Financial graphs (Hadoop)	Precision: 93.6%

[redacted]

[redacted]

[redacted]

[redacted]

```
Dataset shape: (568630, 31)
   id        V1        V2        V3        V4        V5        V6        V7  \
0   0 -0.260648 -0.469648  2.496266 -0.083724  0.129681  0.732898  0.519014
1   1  0.985100 -0.356045  0.558056 -0.429654  0.277140  0.428605  0.406466
2   2 -0.260272 -0.949385  1.728538 -0.457986  0.074062  1.419481  0.743511
3   3 -0.152152 -0.508959  1.746840 -1.090178  0.249486  1.143312  0.518269
4   4 -0.206820 -0.165280  1.527053 -0.448293  0.106125  0.530549  0.658849

          V8        V9  ...       V21       V22       V23       V24       V25  \
0 -0.130006  0.727159  ... -0.110552  0.217606 -0.134794  0.165959  0.126280
1 -0.133118  0.347452  ... -0.194936 -0.605761  0.079469 -0.577395  0.190090
2 -0.095576 -0.261297  ... -0.005020  0.702906  0.945045 -1.154666 -0.605564
3 -0.065130 -0.205698  ... -0.146927 -0.038212 -0.214048 -1.893131  1.003963
4 -0.212660  1.049921  ... -0.106984  0.729727 -0.161666  0.312561 -0.414116

        V26       V27       V28    Amount  Class
0 -0.434824 -0.081230 -0.151045  17982.10      0
1  0.296503 -0.248052 -0.064512   6531.37      0
2 -0.312895 -0.300258 -0.244718   2513.54      0
3 -0.515950 -0.165316  0.048424   5384.44      0
4  1.071126  0.023712  0.419117  14278.97      0

[5 rows x 31 columns]
```

[redacted]

```
Class
0    284315
1    284315
Name: count, dtype: int64

Columns after preprocessing:
['V1', 'V2', 'V3', 'V4', 'V5', 'V6', 'V7', 'V8', 'V9', 'V10', 'V11', 'V12', 'V13', 'V14', 'V15', 'V16', 'V17', 'V18', 'V19', 'V20', 'V21', 'V22', 'V23', 'V24', 'V25', 'V26', 'V27', 'V28', 'Class', 'Scaled_A
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

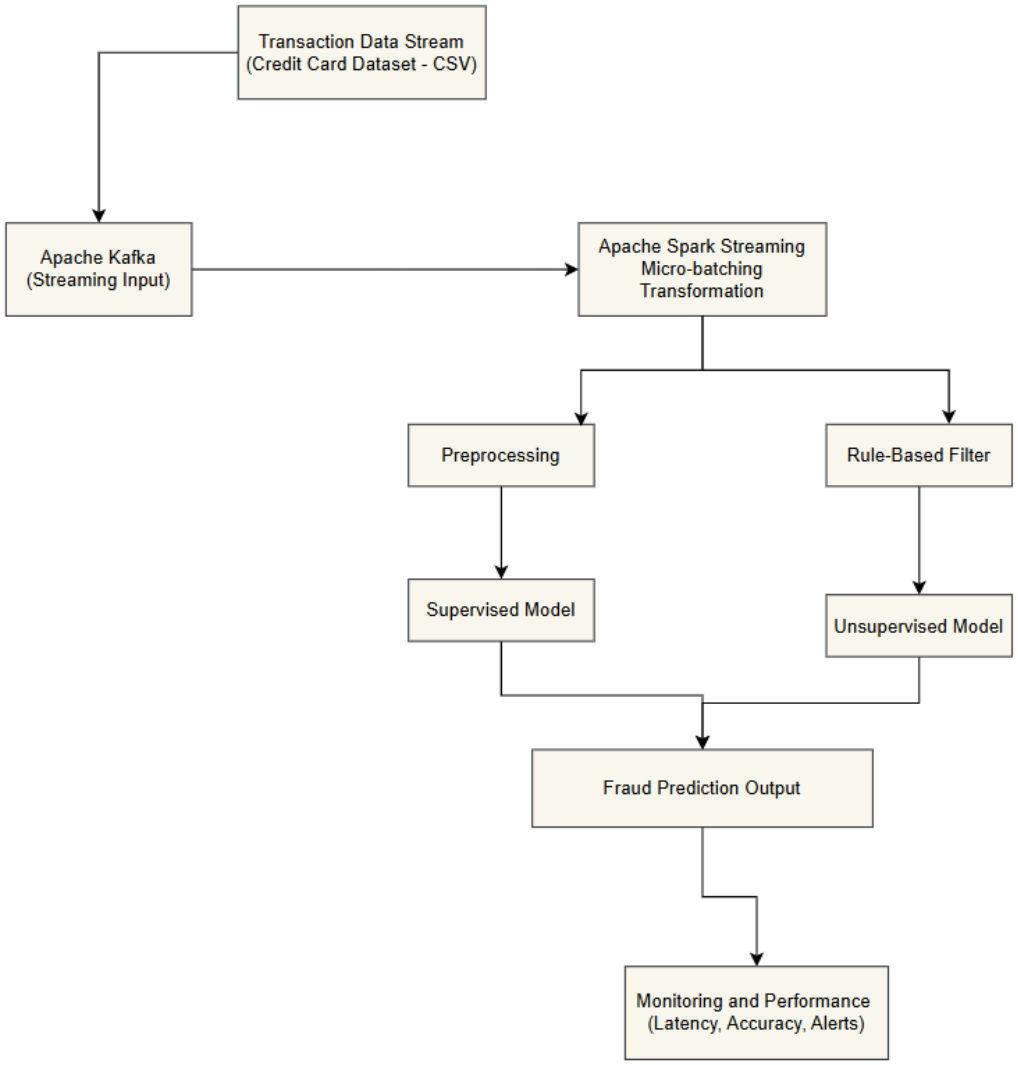
[redacted]

[redacted]

[redacted]

[redacted]

[redacted]


Transaction Data Stream
(Credit Card Dataset - CSV)
Apache Kafka
(Streaming Input)
Apache Spark Streaming
Micro-batching
Transformation
Preprocessing
Rule-Based Filter
Supervised Model
Unsupervised Model
Fraud Prediction Output
Monitoring and Performance
(Latency, Accuracy, Alerts)

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```python
# STEP 1: Install Kaggle and dependencies
!pip install -q kaggle

# STEP 2: Create kaggle.json from credentials
import os
import json

kaggle_token = {
    "username": "your_kaggle_username",
    "key": "your_kaggle_api_key"
}

os.makedirs("/root/.kaggle", exist_ok=True)
with open("/root/.kaggle/kaggle.json", "w") as f:
    json.dump(kaggle_token, f)

# STEP 3: Set proper permission
!chmod 600 /root/.kaggle/kaggle.json

# STEP 4: Download the correct dataset
!kaggle datasets download -d nelgiriyewithana/credit-card-fraud-detection-dataset-2023

# STEP 5: Unzip the dataset
!unzip -q credit-card-fraud-detection-dataset-2023.zip

# STEP 6: Confirm downloaded file
!ls -l
```

[redacted]

[redacted]

[redacted]

```python
# STEP 2.2: Load the dataset
df = pd.read_csv("creditcard_2023.csv")

# STEP 2.3: Initial exploration
print("Dataset shape:", df.shape)
print(df.head())
print(df['Class'].value_counts())
```

[redacted]

```
Dataset shape: (568630, 31)
   id        V1        V2        V3        V4        V5        V6        V7  \
0   0 -0.260648 -0.469648  2.496266 -0.083724  0.129681  0.732898  0.519014
1   1  0.985100 -0.356045  0.558056 -0.429654  0.277140  0.428605  0.406466
2   2 -0.260272 -0.949385  1.728538 -0.457986  0.074062  1.419481  0.743511
3   3 -0.152152 -0.508959  1.746840 -1.090178  0.249486  1.143312  0.518269
4   4 -0.206820 -0.165280  1.527053 -0.448293  0.106125  0.530549  0.658849

         V8        V9  ...       V21       V22       V23       V24       V25  \
0 -0.130006  0.727159  ... -0.110552  0.217606 -0.134794  0.165959  0.126280
1 -0.133118  0.347452  ... -0.194936 -0.605761  0.079469 -0.577395  0.190090
2 -0.095576 -0.261297  ... -0.005020  0.702906  0.945045 -1.154666 -0.605564
3 -0.065130 -0.205698  ... -0.146927 -0.038212 -0.214048 -1.893131  1.003963
4 -0.212660  1.049921  ... -0.106984  0.729727 -0.161666  0.312561 -0.414116

        V26       V27       V28    Amount  Class
0 -0.434824 -0.081230 -0.151045  17982.10      0
1  0.296503 -0.248052 -0.064512   6531.37      0
2 -0.312895 -0.300258 -0.244718   2513.54      0
3 -0.515950 -0.165316  0.048424   5384.44      0
4  1.071126  0.023712  0.419117  14278.97      0
```

[redacted]

[redacted]

[redacted]

[redacted]

```
# STEP 2.4: Drop unnecessary columns
if 'id' in df.columns:
    df.drop(columns=["id"], inplace=True)
```

```
# STEP 2.5: Feature scaling for 'Amount'
scaler = MinMaxScaler()
df['Scaled_Amount'] = scaler.fit_transform(df[['Amount']])
df.drop(['Amount'], axis=1, inplace=True)
```

```python
# STEP 2.6: Simulate transaction velocity (since no 'Time' column)
df['Trans_Per_Second'] = np.random.uniform(0.01, 10, size=len(df))
```

[redacted]

```python
# STEP 2.7: Check final column names
print("\nColumns after preprocessing:")
print(df.columns.tolist())

# STEP 2.8: Split features and target
X = df.drop(['Class'], axis=1)
y = df['Class']
```

[redacted]

```python
# STEP 2.9: Addressing class imbalance with SMOTE
print("\nBefore SMOTE:")
print(y.value_counts())

sm = SMOTE(random_state=42)
X_resampled, y_resampled = sm.fit_resample(X, y)

print("\nAfter SMOTE:")
print(pd.Series(y_resampled).value_counts())
```

[redacted]

```python
# STEP 2.10: Final train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X_resampled, y_resampled, test_size=0.2, random_state=42, stratify=y_resampled)

print(f"\n Train shape: {X_train.shape}, Test shape: {X_test.shape}")
```

[redacted]

[redacted]

```
[ ] # STEP 3.1: Import ML libraries
    from sklearn.ensemble import RandomForestClassifier
    from sklearn.linear_model import LogisticRegression
    from xgboost import XGBClassifier
    from sklearn.metrics import classification_report, roc_auc_score
    import joblib

    # STEP 3.2: Sample 100,000 rows for training
    sample_size = 100000
    X_train_small = X_train.sample(n=sample_size, random_state=42)
    y_train_small = y_train.loc[X_train_small.index]

    # Sample 20,000 for testing
    X_test_small = X_test.sample(n=20000, random_state=42)
    y_test_small = y_test.loc[X_test_small.index]

    print(f" Training shape: {X_train_small.shape}, Testing shape: {X_test_small.shape}")
```

[redacted]

```
# STEP 3.3: Train Random Forest
rf_model = RandomForestClassifier(n_estimators=50, n_jobs=-1, random_state=42)
rf_model.fit(X_train_small, y_train_small)
rf_preds = rf_model.predict(X_test_small)
print("\n Random Forest Performance:")
print(classification_report(y_test_small, rf_preds))
print("AUC-ROC:", roc_auc_score(y_test_small, rf_model.predict_proba(X_test_small)[:, 1]))
joblib.dump(rf_model, 'random_forest_model.joblib')
```

```python
# STEP 3.4: Train XGBoost
xgb_model = XGBClassifier(use_label_encoder=False, eval_metric='logloss', n_estimators=50, random_state=42)
xgb_model.fit(X_train_small, y_train_small)
xgb_preds = xgb_model.predict(X_test_small)
print("\n XGBoost Performance:")
print(classification_report(y_test_small, xgb_preds))
print("AUC-ROC:", roc_auc_score(y_test_small, xgb_model.predict_proba(X_test_small)[:, 1]))
joblib.dump(xgb_model, 'xgboost_model.joblib')
```

```python
# STEP 3.5: Train Logistic Regression
lr_model = LogisticRegression(max_iter=1000, class_weight='balanced', random_state=42)
lr_model.fit(X_train_small, y_train_small)
lr_preds = lr_model.predict(X_test_small)
print("\n Logistic Regression Performance:")
print(classification_report(y_test_small, lr_preds))
print("AUC-ROC:", roc_auc_score(y_test_small, lr_model.predict_proba(X_test_small)[:, 1]))
joblib.dump(lr_model, 'logistic_regression_model.joblib')
```

```
# STEP 3.6: Confirm model files
print("\n Models saved:")
!ls -lh *.joblib
```

```
[ ] import joblib
    import time
    import random

    # Load the trained model (choose any one)
    model = joblib.load("xgboost_model.joblib")

    # Simulate real-time mini-stream (100 records)
    stream_data = X_test.sample(n=100, random_state=42)
    true_labels = y_test.loc[stream_data.index]

    print(" Starting Real-Time Stream Simulation...\n")
    for i in range(len(stream_data)):
        # Simulate delay like real streaming
        time.sleep(random.uniform(0.05, 0.1))  # ~50-100 ms delay

        # Get single transaction (reshape for sklearn)
        transaction = stream_data.iloc[i].values.reshape(1, -1)
        prediction = model.predict(transaction)[0]
        prob = model.predict_proba(transaction)[0][1]

        # Print prediction
        print(f"Txn {i+1:03d}: Fraud={bool(prediction)}, Probability={prob:.4f}, True={true_labels.iloc[i]}")
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```
[ ] stream_data.to_csv("stream_simulated_transactions.csv", index=False)
```

[redacted]

[redacted]

[redacted]

```
Dataset shape: (568630, 31)
   id        V1        V2        V3        V4        V5        V6        V7  \
0   0 -0.260648 -0.469648  2.496266 -0.083724  0.129681  0.732898  0.519014
1   1  0.985100 -0.356045  0.558056 -0.429654  0.277140  0.428605  0.406466
2   2 -0.260272 -0.949385  1.728538 -0.457986  0.074062  1.419481  0.743511
3   3 -0.152152 -0.508959  1.746840 -1.090178  0.249486  1.143312  0.518269
4   4 -0.206820 -0.165280  1.527053 -0.448293  0.106125  0.530549  0.658849

          V8        V9  ...       V21       V22       V23       V24       V25  \
0 -0.130006  0.727159  ... -0.110552  0.217606 -0.134794  0.165959  0.126280
1 -0.133118  0.347452  ... -0.194936 -0.605761  0.079469 -0.577395  0.190090
2 -0.095576 -0.261297  ... -0.005020  0.702906  0.945045 -1.154666 -0.605564
3 -0.065130 -0.205698  ... -0.146927 -0.038212 -0.214048 -1.893131  1.003963
4 -0.212660  1.049921  ... -0.106984  0.729727 -0.161666  0.312561 -0.414116

        V26       V27       V28    Amount  Class
0 -0.434824 -0.081230 -0.151045  17982.10      0
1  0.296503 -0.248052 -0.064512   6531.37      0
2 -0.312895 -0.300258 -0.244718   2513.54      0
3 -0.515950 -0.165316  0.048424   5384.44      0
4  1.071126  0.023712  0.419117  14278.97      0

[5 rows x 31 columns]
```

```
Training shape: (100000, 30), Testing shape: (20000, 30)
```

```
Random Forest Performance:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00      9995
           1       1.00      1.00      1.00     10005

    accuracy                           1.00     20000
   macro avg       1.00      1.00      1.00     20000
weighted avg       1.00      1.00      1.00     20000
```

AUC-ROC: 0.9998767499691875

XGBoost Performance:

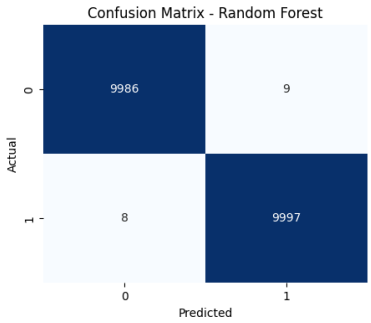
	precision	recall	f1-score	support
0	1.00	1.00	1.00	9995
1	1.00	1.00	1.00	10005
accuracy			1.00	20000
macro avg	1.00	1.00	1.00	20000
weighted avg	1.00	1.00	1.00	20000

AUC-ROC: 0.9999011849752962

Logistic Regression Performance:

	precision	recall	f1-score	support
0	0.95	0.98	0.97	9995
1	0.98	0.95	0.97	10005
accuracy			0.97	20000
macro avg	0.97	0.97	0.97	20000
weighted avg	0.97	0.97	0.97	20000

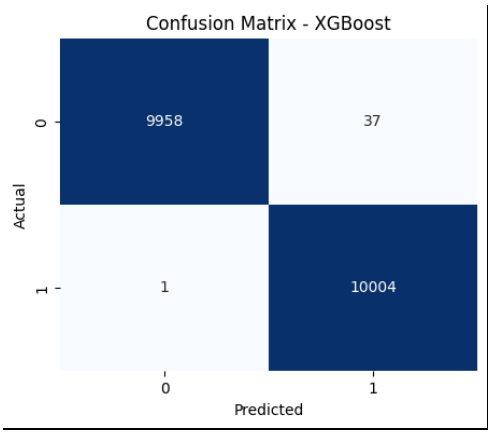
AUC-ROC: 0.9934971833742958



[redacted]

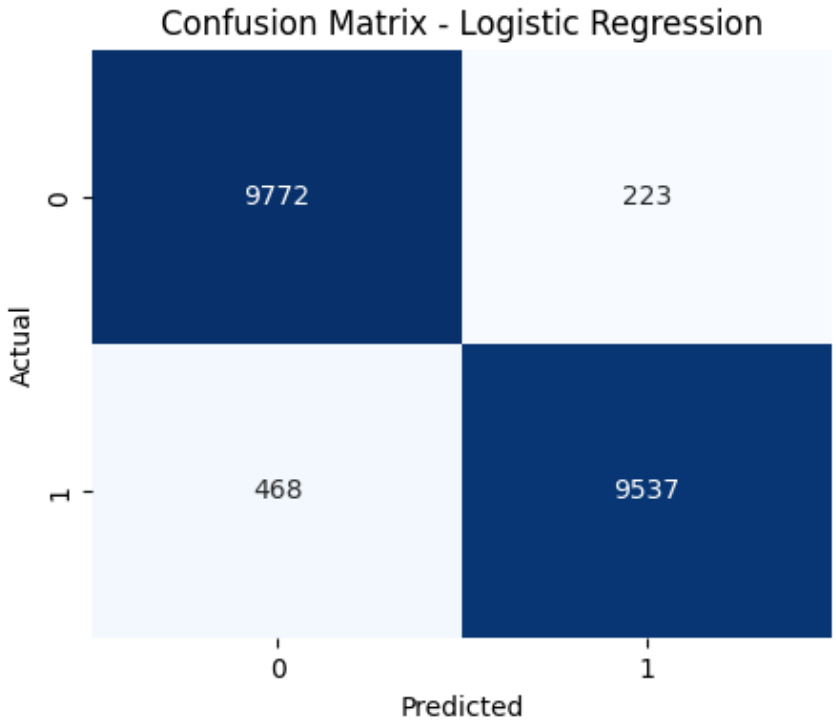
[redacted]

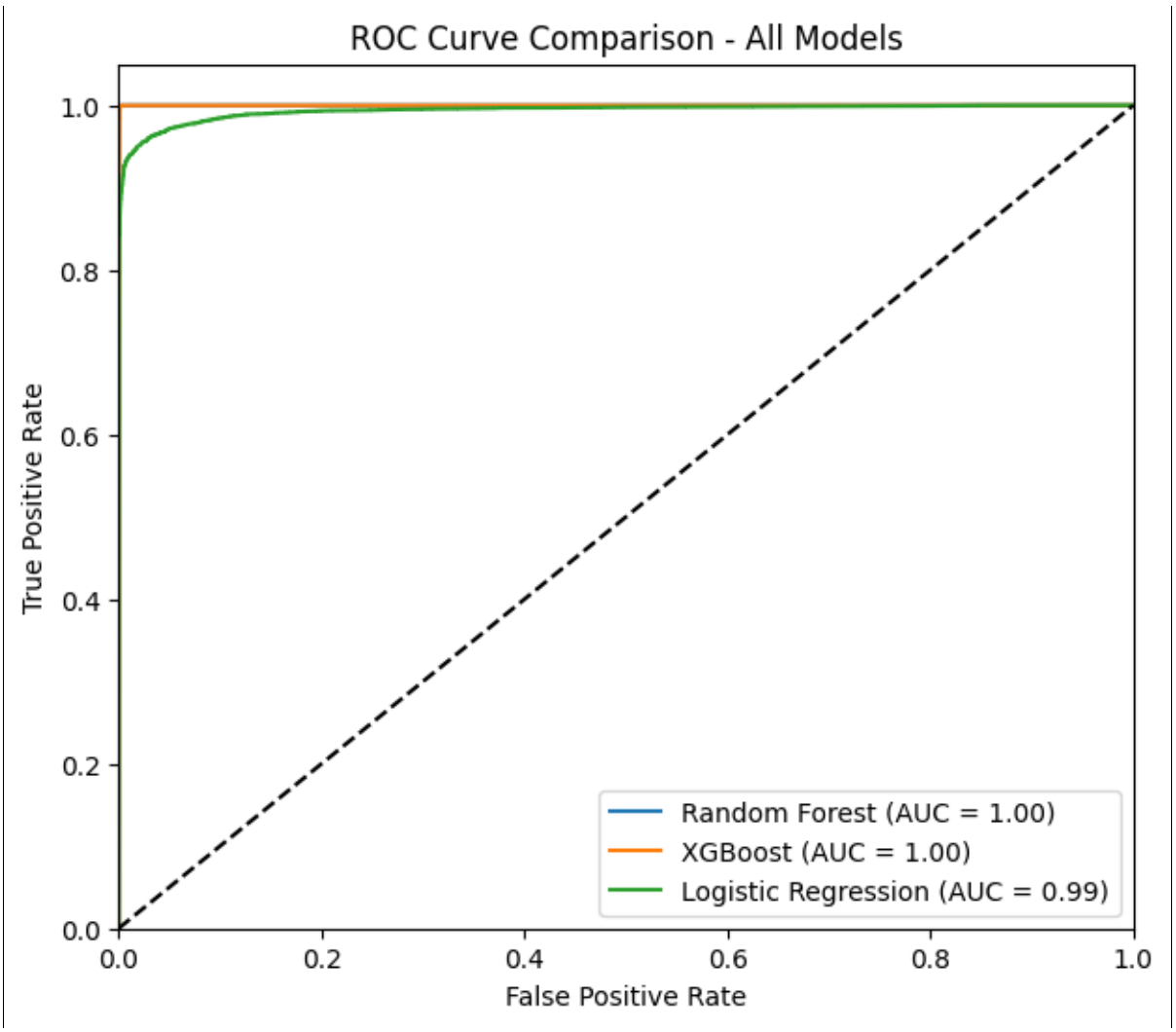
[redacted]

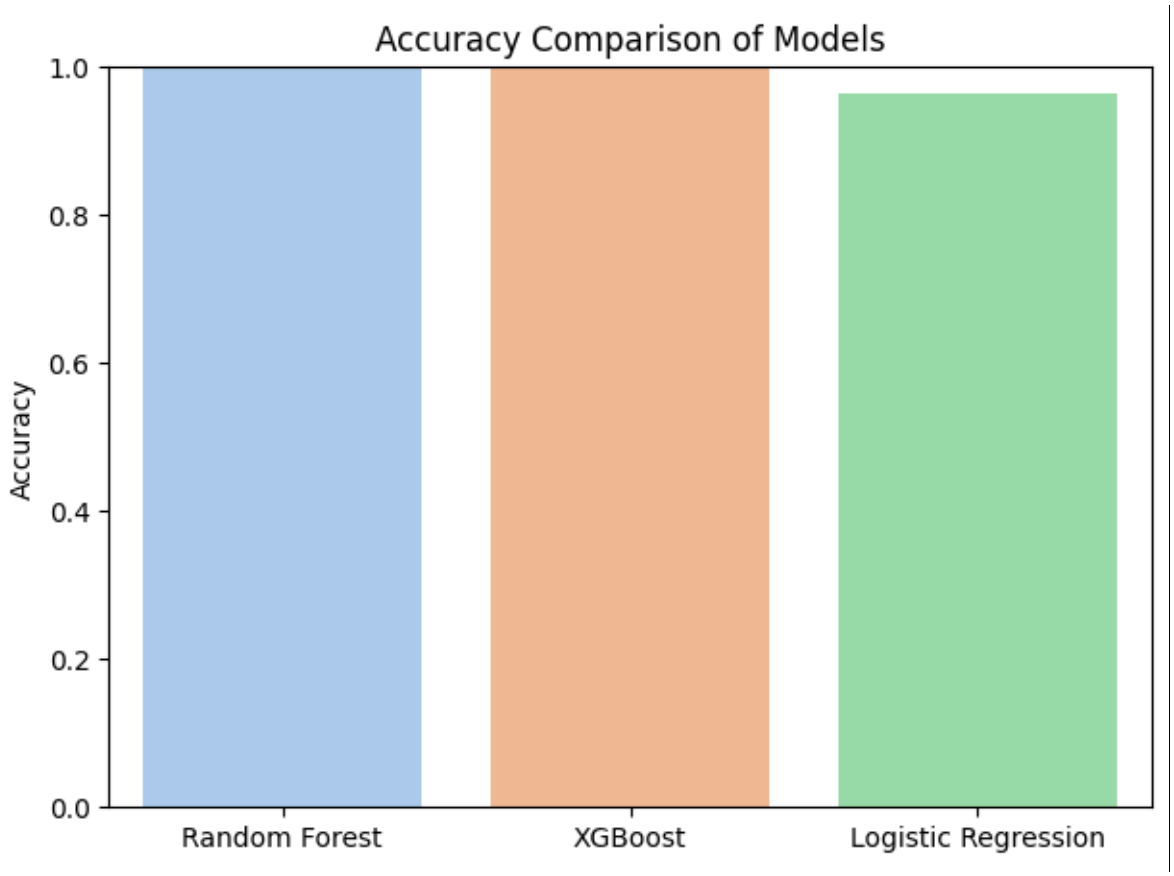


[redacted]

[redacted]


Confusion Matrix - Logistic Regression
9772
223
468
9537
0
1
Actual
0
1
Predicted


ROC Curve Comparison - All Models
True Positive Rate
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0
Random Forest (AUC = 1.00)
XGBoost (AUC = 1.00)
Logistic Regression (AUC = 0.99)


Accuracy Comparison of Models
Accuracy
1.0
0.8
0.6
0.4
0.2
0.0
Random Forest
XGBoost
Logistic Regression

[redacted]

```
Starting Real-Time Stream Simulation...

Txn 001: Fraud=False, Probability=0.0007, True=0
Txn 002: Fraud=False, Probability=0.0002, True=0
Txn 003: Fraud=True, Probability=0.9998, True=1
Txn 004: Fraud=True, Probability=0.9998, True=1
Txn 005: Fraud=False, Probability=0.0002, True=0
Txn 006: Fraud=True, Probability=1.0000, True=1
Txn 007: Fraud=False, Probability=0.0003, True=0
Txn 008: Fraud=True, Probability=0.9998, True=1
Txn 009: Fraud=True, Probability=0.9999, True=1
Txn 010: Fraud=False, Probability=0.0001, True=0
Txn 011: Fraud=True, Probability=1.0000, True=1
Txn 012: Fraud=True, Probability=0.6098, True=1
Txn 013: Fraud=False, Probability=0.0010, True=0
Txn 014: Fraud=True, Probability=0.9993, True=1
Txn 015: Fraud=False, Probability=0.0000, True=0
Txn 016: Fraud=False, Probability=0.0011, True=0
Txn 017: Fraud=False, Probability=0.0161, True=0
Txn 018: Fraud=True, Probability=0.9994, True=1
Txn 019: Fraud=True, Probability=0.9999, True=1
Txn 020: Fraud=True, Probability=0.9680, True=1
Txn 021: Fraud=False, Probability=0.0001, True=0
Txn 022: Fraud=True, Probability=0.9998, True=1
Txn 023: Fraud=True, Probability=0.9999, True=1
Txn 024: Fraud=False, Probability=0.0086, True=0
Txn 025: Fraud=True, Probability=0.9999, True=1
Txn 026: Fraud=True, Probability=1.0000, True=1
Txn 027: Fraud=False, Probability=0.0002, True=0
Txn 028: Fraud=True, Probability=0.9994, True=1
Txn 029: Fraud=True, Probability=1.0000, True=1
Txn 030: Fraud=True, Probability=0.9999, True=1
Txn 031: Fraud=True, Probability=0.9999, True=1
Txn 032: Fraud=True, Probability=1.0000, True=1
Txn 033: Fraud=True, Probability=1.0000, True=1
Txn 034: Fraud=True, Probability=0.9998, True=1
Txn 035: Fraud=True, Probability=0.9999, True=1
Txn 036: Fraud=False, Probability=0.0007, True=0
Txn 037: Fraud=True, Probability=0.9999, True=1
Txn 038: Fraud=False, Probability=0.0000, True=0
Txn 039: Fraud=False, Probability=0.0001, True=0
Txn 040: Fraud=True, Probability=0.9999, True=1
Txn 041: Fraud=True, Probability=0.9993, True=1
Txn 042: Fraud=False, Probability=0.0001, True=0
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]